

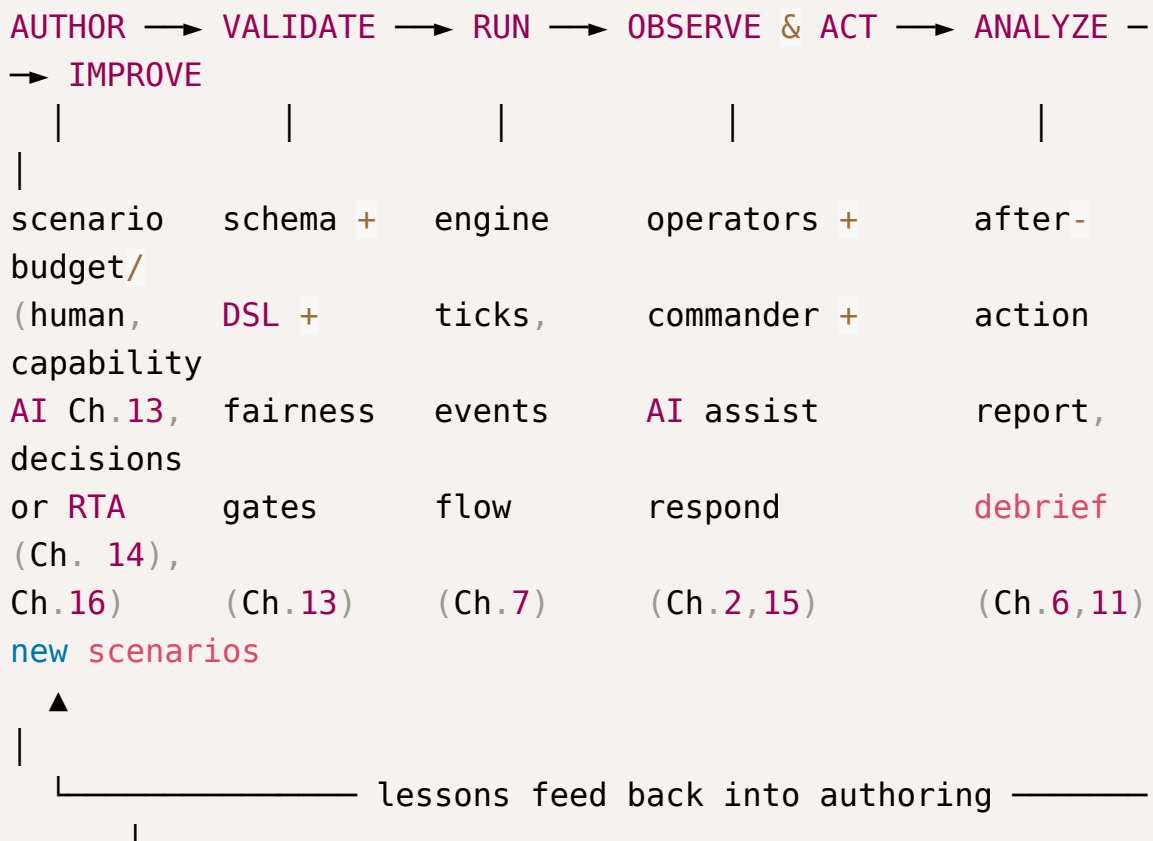


Chapter 17 — Project & Platform Workflows

This chapter maps every major flow in DOIP end to end — how the platform operates at runtime, how each user journey unfolds, and how the project itself is developed and released. Use it as the connective tissue between chapters: each workflow references the chapter that specifies its parts.

17.1 The Master Workflow — From Scenario to Debrief

Everything in DOIP follows one macro-cycle:



Every loop through this cycle produces durable artifacts: an event log (replayable), an after-action report, performance scores, and — via Chapter 14 — evidence for preparedness investment.

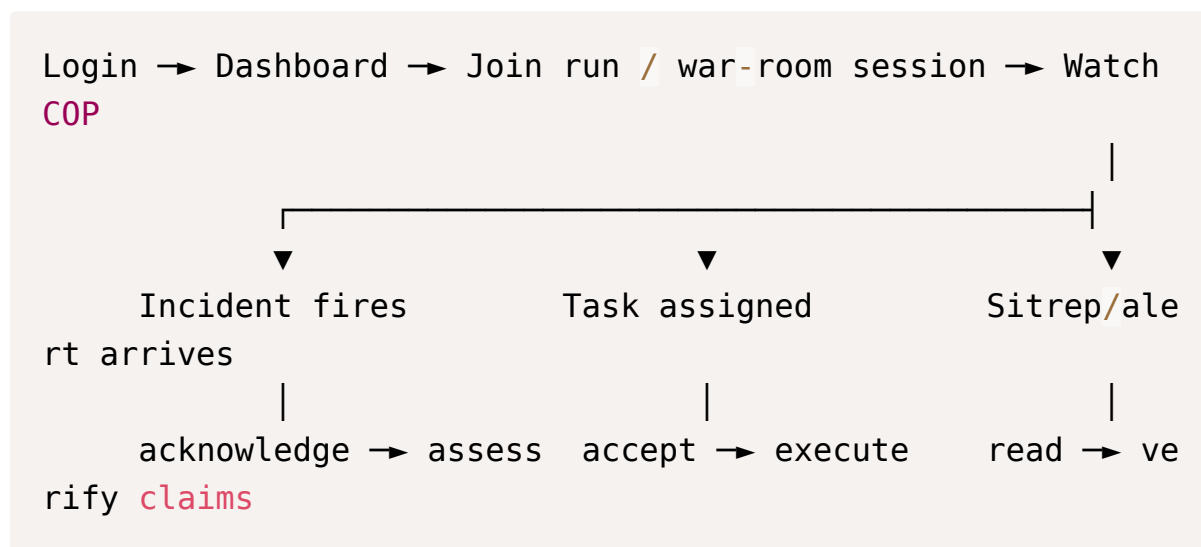
17.2 Runtime Workflow — One Tick of the Platform

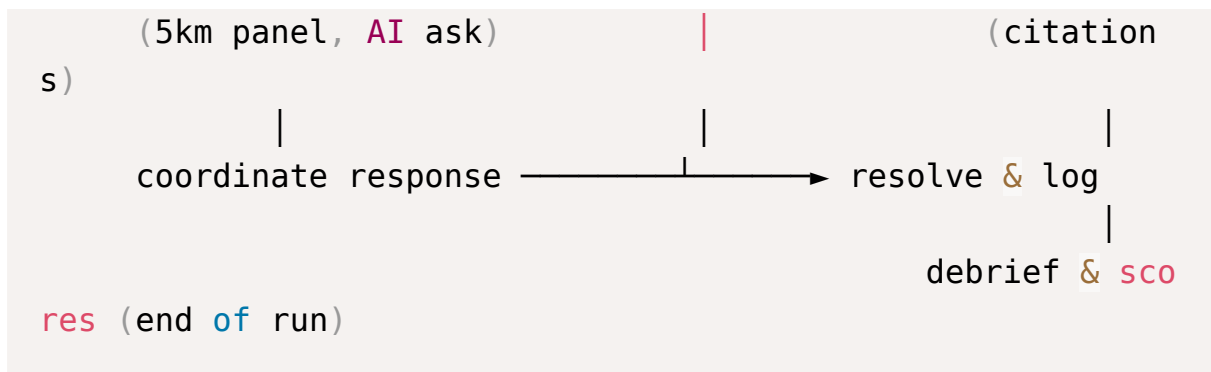
What happens continuously while a run is live (the “heartbeat”):

1. **Tick** — the engine advances sim time, moves entities, evaluates scenario rules and RTA injects, updates consumption/vitals/weather (Ch. 7).
2. **Emit** — state changes become typed events; positions become lossy 4 Hz frames (Ch. 6).
3. **Persist** — events append to the `events` table; projections update (Ch. 5).
4. **Publish** — events fan out via Redis/Kafka to the WS gateway and service consumers (Ch. 3).
5. **Render** — each client's reducers update map, feeds, and widgets, filtered by role/sector scope (Ch. 15).
6. **Assist** — the AI service buffers events for the next sitrep; the assessor updates trainee scores (Ch. 8, 16).
7. **Record** — analytics appends telemetry for forecasting and after-action metrics (Ch. 11, 14).

Latency budget across steps 1 → 5: **p95 < 1 second** (NFR-1).

17.3 User Workflow — Operator's Session (the daily loop)





Key rule embedded in the UX: **every AI output is a suggestion with citations; the operator verifies and decides** (Ch. 1, principle 4).

17.4 Incident Lifecycle Workflow

detected → created → (auto-prioritized) → acknowledged → responded → resolved → analyzed

Stage	Actor	System support
Detection	Engine / CV pipeline / RTA inject	Sensor fusion, YOLO detections auto-create candidates (Ch. 2, 8)
Creation & priority	Engine	Severity scoring, map ping, feed entry
Acknowledgment	Operator	"Units within 5 km" ranking, SOP lookup via RAG
Response	Operator/Planner	Task assignment (Ch. 15), mission launch if needed (Ch. 2 FR-6)
Resolution	Operator	Resolution notes → event log
Analysis	Analytics + instructor	Response-time metrics, debrief timeline (Ch. 11)

Every transition is an event — so the entire lifecycle is replayable and auditable by construction.

17.5 Mission Planning Workflow

1. Planner draws waypoints/survey zone on the map.
2. System builds the routing graph, applies constraints (restricted zones, weather costs).
3. **Optimize** — A* route + coverage path; comparison vs naive route shown.
4. **Feasibility check** — battery/duration verdict with explanations; infeasible plans say *why*.

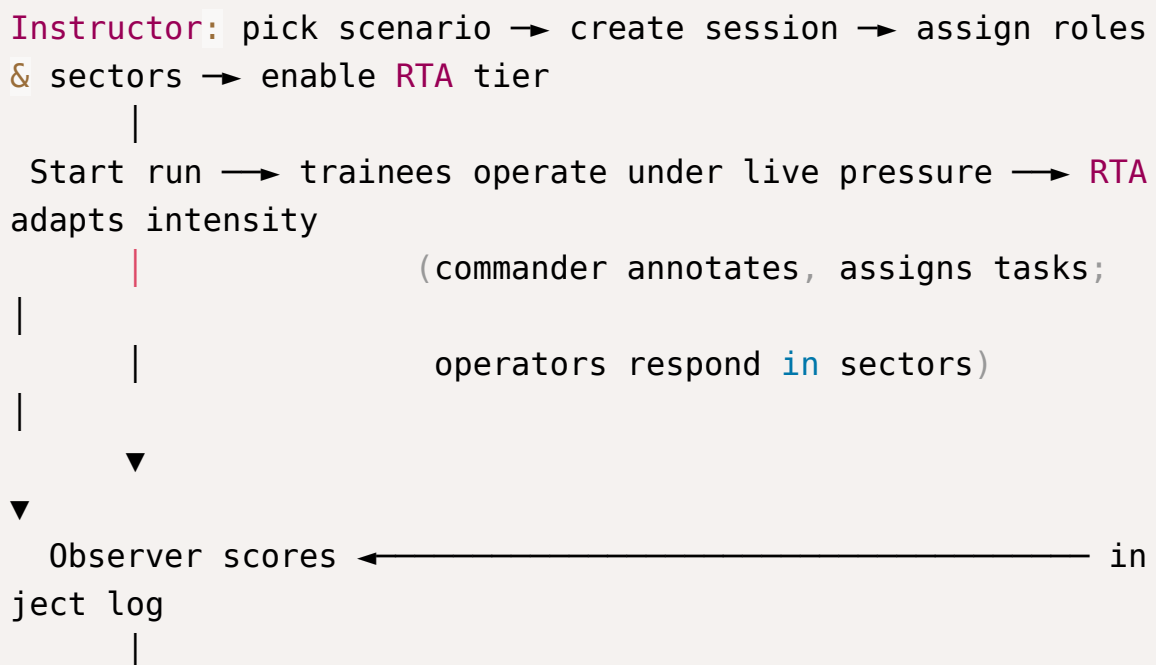
5. Launch → engine executes → `WaypointReached` / `MissionCompleted` events stream live.
6. Post-mission: actual-vs-predicted ETA and consumption feed the forecasting models (Ch. 8).

17.6 AI Workflows (three distinct pipelines)

Workflow	Trigger	Path	Human control point
Sitrep (Ch. 8)	Timer or on-demand	collect → cluster → draft → grounding check → publish	Read with citations; regenerate
RAG answer (Ch. 8)	Operator question	route → retrieve → generate with citations	Verify sources before acting
What-If Theater (Ch. 13)	NL prompt	LLM author → Validation Gate → confirm/edit → headless run → animated playback + narration	Mandatory approve/edit step before any run

All three share one invariant: **no ungrounded claim reaches the user unlabeled.**

17.7 Training Exercise Workflow (War-Room + Red-Team)



```
End run → debrief replay (annotations + injects + score timeline) → certification metrics
```

Repeatability guarantee: rerunning the same scenario/seed with RTA produces the identical exercise — enabling fair comparison across trainee cohorts (Ch. 16 §16.2).

17.8 Preparedness Investment Workflow (Ch. 14)

```
import budget/inventory → run scenario battery → gap register → optimizer portfolios → validation re-simulation → evidence-backed recommendation report → (human decision) → re-import updated inventory → measure improvement
```

This is the platform's slowest loop (weeks, not seconds) — and the one that closes the “prepare better, save lives” mission: simulated evidence → investment decision → measurably better simulated outcomes.

17.9 Development Workflow (how the project itself is built)

```
Idea/requirement → GitHub issue (phase-tagged) → short-lived branch
    |
    code + tests (Definition of Done, Ch. 9 §9.11)
    |
    PR → CI: lint → typecheck → unit/integration → golden-run suite
    |
    vals, E2E | (nightly: AI evals, E2E)
    review ← regenerated golden fixtures if engine behavior changed
    |
    merge to main → phase exit criteria met? → tag v<phase>.<sprint>
    |
    demo recording + changelog + release
```

Cadence rules (from Ch. 9): one phase at a time, every phase ends with a tagged release and demo, scope is cut — never deadlines extended — when a

sprint overruns by 50%.

17.10 Data Lifecycle Workflow

Stage	What happens	Retention
Hot (live run)	Events + projections in Postgres; positions batched	Full fidelity
Warm (completed run)	Run archive: events + scenario + seed; after-action report generated	Full fidelity, exportable
Cold	Position downsampling, old runs archived to MinIO	Summaries + archives
Models	Forecast/CV models versioned in MLflow registry; every prediction carries model lineage	All versions
Audit	<code>audit_logs</code> • AI provenance (which model, which events)	Never pruned in v1

17.11 Workflow ↔ Chapter Map (quick index)

Workflow	Specified in
Tick/event heartbeat	Ch. 3, 6, 7
Operator & incident flows	Ch. 2, 11
Mission planning	Ch. 2 (FR-6), 7
AI pipelines	Ch. 8, 13
Training exercises	Ch. 15, 16
Investment analysis	Ch. 14
Development & release	Ch. 9, 10
Data lifecycle	Ch. 5, 10